

SOLO HACKATHON CHALLENGE · MLOPS TRACK

# SHIP/60

## The Production Gauntlet

Ship one production-grade AI service in 60 days — containerised, cloud-deployed, monitored, traced, and provisioned by code. Learn every layer by building it, not watching it.

**60**

DAYS

**~1 hr**

PER DAY

**970**

MAX POINTS

**6+2**

LEVELS / BUFFERS

Build: **YieldGuard** — a predict-and-explain AI service  
Theme: Semiconductor / Manufacturing ML portfolio

Challenger: \_\_\_\_\_  
Start: \_\_\_\_\_ · Ship: \_\_\_\_\_

# Your Mission

---

Single-handedly take a machine-learning model from a notebook to a live, monitored, cloud-deployed service — and be able to explain every decision in between. This is the exact gap between an "automation builder" profile and the MLOps / AI Engineer roles you're targeting in the German market.

## The Product — YieldGuard

A semiconductor-themed AI service you can put in front of Infineon, GlobalFoundries, Bosch, or ZEISS. Built once, then layered with a new skill at every level.

### HEAD A · CORE BUILD

#### The Predictor

A tabular ML model predicting pass/fail or a yield metric from process data. Your classic-MLOps spine.

### HEAD B · ADD LATER

#### The Explainer

A RAG assistant over technical docs. You already know RAG — this is a bolt-on, not a mountain.

**Data — zero hunting required.** Load real semiconductor yield data in one line: `fetch_ucirepo(id=179)` (UCI SECOM, 1567 rows, pre-labelled pass/fail, CC BY 4.0). No download, no labelling, no synthesis. The only touch is a one-time ~30-min cleanup (impute NaNs, keep top ~20 features) — and that itself becomes a CV line: handling messy, high-dimensional, class-imbalanced real-world data. Model accuracy is irrelevant; the infrastructure around it is the skill you're buying.

## The Floor — read this when you want to quit

If everything goes sideways — busy job weeks, application crunch, low energy — this is the minimum that still transforms your CV:

**Kubernetes (real model deployed) + AZ-900 (passed) + one Grafana dashboard.**

That alone clears the screening filter on most German MLOps/AI roles. Everything past it is upside, not obligation.

**Hitting the Floor is a success, not a consolation prize.**

# How The Challenge Runs

## The Rhythm

- **~60 min per session:** 10 min re-read → 45 min build → 5 min commit + 1-line log.
- **4–5 sessions a week.** Two rest days are mandatory, not optional.
- **Bad day?** 25 minutes of anything counts. Showing up beats the streak.
- **Missed a day?** Let it go — never double up. Buffer weeks exist for this.

## The Laws

1. **Learn, then build.** Each level opens with a short concept phase + a readiness gate — understand the basics first, *then* implement to cement them.
2. **Commit daily.** Every session ends with `git push`.
3. **Break it on purpose** once per level, then fix it.
4. **One tool per layer.** No shiny-tool hopping mid-level.
5. **The Understanding Test is the gate** — not "does it run".

### The Anti-Cheat Clause

If you copy-paste a config you can't explain line-by-line, you haven't completed that day. You're allowed to copy — you're *not* allowed to not understand. Re-read until you could rebuild it from memory. **You only unlock a level's points once you've passed its ① Learn readiness gate** — but keep the concept phase tight: it's about the *why* and the vocabulary, not drowning in hours of video. **Points are earned for understanding, never for running.**

# Levels & Points

LEVEL 0 · DAYS 1-3

## Base Camp

CORE

40 pts

Set the stage so you never waste a future session on setup.

### ① LEARN FIRST · ~1 SESSION

**Grasp:** what a container actually is (your app + all its dependencies, isolated and portable); what a REST endpoint does; the difference between a *model file* and a *running model service*.

**Study:** [FastAPI docs — First Steps](#) [Docker — Get Started \(overview\)](#)

**Ready to build when you can explain:** why you containerise instead of "it works on my machine", and what actually happens when something POSTs to `/predict`.

### ② THEN BUILD

- ☐ Create one GitHub repo with a README skeleton mapping all 8 levels. +10
- ☐ Train a dead-simple baseline model on your dataset. Accuracy doesn't matter. +15
- ☐ Wrap it in a FastAPI `/predict` endpoint and a stub `/ask` endpoint. Dockerise it. +15

**Tools:** [Python](#) [FastAPI](#) [Docker](#) [scikit-learn](#) [Git](#)

**Hint:** Don't gold-plate the model. A 5-line classifier is perfect — the entire challenge is about what wraps around it.

**Understanding test:** Why does a model need to be wrapped in an API at all? What's the difference between a *model file* and a *model service*?

Your #1 CV gap. It gets the most time on purpose — don't rush past it.

### ① LEARN FIRST · ~3 SESSIONS

**Grasp:** why orchestration exists (self-healing, scaling, declarative desired-state); the core objects — **Pod**, **Deployment**, **Service**, **ConfigMap**, **Namespace**; the node/control-plane mental model; what Helm packages and why.

**Study:** [kubernetes.io](https://kubernetes.io) — "Kubernetes Basics" interactive tutorial

[KodeKloud](#) — free Kubernetes labs

"Kubernetes in 100 Seconds" for the 2-min mental model

**Ready to build when you can explain:** what Kubernetes solves over plain Docker, and the difference between a Pod, a Deployment, and a Service — *before* you touch `kubectl`.

### ② THEN BUILD

- ☐ Install Minikube locally (no cloud cost yet). +20
- ☐ Deploy the Predictor: **Pod** → **Deployment** → **Service**. +50
- ☐ Add a ConfigMap + Namespace. Live in `kubectl`: get, describe, logs, apply, port-forward. +40
- ☐ Package the whole thing as a basic **Helm chart**. +50

**Boss +40:** Scale to 3 replicas, then `kubectl delete pod` one of them. Watch Kubernetes self-heal — and explain what just happened.

#### Tools:

[Kubernetes](#)
[Minikube](#)
[kubectl](#)
[Helm](#)

**Hint:** Use KodeKloud's free labs to drill `kubectl`, but deploy *your own* container, not their demo. Do **not** start CKA prep yet — understanding first, cert later.

**Understanding test:** Why a Deployment instead of a bare Pod? What problem does a Service actually solve? Why Helm over raw YAML?

Azure dominates German enterprise. Get off your laptop and onto the cloud.

#### ① LEARN FIRST · THE AZ-900 STUDY IS THE LEARN PHASE

**Grasp:** cloud service models (IaaS/PaaS/SaaS), core Azure building blocks, and — most important for you — where **AKS**, **ACR**, and **Azure ML** each fit in a deployment.

**Study:** Microsoft Learn — **AZ-900 Azure Fundamentals path** (free, official, exam-aligned)

**Ready to build when you can explain:** the difference between Azure ML, AKS, and ACR, and when you'd reach for each.

#### ② THEN BUILD

- ☐ Study **AZ-900** via Microsoft Learn (built into the week). +30
- ☐ Push your image to **Azure Container Registry**. +30
- ☐ Stand up **AKS** and redeploy your Helm chart there. +60

**Boss +80:** Book and **pass the AZ-900 exam** (€165). Go at 80% ready — don't wait for 100%. This single badge changes how your CV is screened.

**Tools:** Azure AKS ACR Azure ML Microsoft Learn

**Hint:** Set an Azure **spending alert on day one** and run `az aks stop` whenever you're not actively using the cluster. AKS is not free.

**Understanding test:** When would you use Azure ML vs AKS? Why a container registry instead of rebuilding the image everywhere?

Make the invisible visible. This is the "production-grade" signal recruiters look for.

### ① LEARN FIRST · ~2 SESSIONS

**Grasp:** metrics vs logs vs traces; Prometheus' pull model + time-series; what PromQL is; the **four golden signals** (latency, traffic, errors, saturation); what Grafana actually does with the data.

**Study:** Prometheus docs — Overview & Getting Started

Grafana — Fundamentals

Google SRE Book — Ch.6 "Monitoring Distributed Systems"

**Ready to build when you can explain:** why a metric isn't a log, and which 3–4 signals actually matter on a live model.

### ② THEN BUILD

- ☐ Deploy **Prometheus + Grafana** (Docker Compose, then wire to the cluster). +40
- ☐ Instrument 3 metrics: prediction **latency, throughput, error rate**. +40
- ☐ Build one **Grafana dashboard** worth staring at. Add structured Python logging. +40

**Boss +30:** Write one Prometheus alert rule (e.g. error rate > 5% or p95 latency > 500ms) and trigger it on purpose.

**Tools:** Prometheus Grafana Docker Compose

**Hint:** Skip the ELK stack — it's overkill here. Structured Python logging plus three good Prometheus metrics tells the whole production story.

**Understanding test:** What's the difference between logs, metrics, and traces? Why monitor a model beyond "is it up"? →

**Completing this closes the Floor.**

## Week 5 — Buffer Week · 0 pts (by design)

No new content. Pick one: **behind?** catch up calmly · **on track?** polish & rest · **crushed by job/applications?** this is the week that absorbs it, zero guilt. This week is precisely why you won't drop the project. Use it.

Bring the Explainer head to life. You've built RAG at QuickFlows — the new part is just the observability layer.

① **LEARN FIRST** · ~2 SESSIONS (YOU ALREADY HAVE THE RAG BASICS)

**Grasp:** what a **trace** / **span** means for an LLM call; why LLM apps need different monitoring than classic ML; and how you evaluate output with no single right answer (faithfulness, retrieval relevance, answer quality).

**Study:** [Langfuse docs — Tracing & Evaluation](#) [Ragas — RAG evaluation concepts](#)

**Ready to build when you can explain:** what a Langfuse trace captures, and how you'd measure "is this RAG answer good?" without a labelled answer key.

② **THEN BUILD**

☐ Wire the /ask endpoint with a real RAG pipeline. +40

☐ Add **Langfuse** tracing — every query traced end-to-end. +40

☐ Add 2 eval metrics (retrieval quality + latency) and deploy on the same cluster. +40

**Boss +40:** Serve the model with **vLLM** instead of a hosted API and compare latency (appears in real JDs).

**Tools:** [RAG](#) [Langfuse](#) [LangChain](#) [vLLM \(boss\)](#)

**Hint:** This is your most *droppable* core item if you're behind — it's your existing strength, so the marginal learning is lowest. The new muscle is tracing, not RAG.

**Understanding test:** Where does a RAG pipeline fail *silently*, and how would your traces catch it? What does "evaluation" mean with no single right answer?



## Week 7 — Buffer or Stretch · up to 120 pts

**Behind?** Catch up — no new tools. **Ahead and energised?** Take the stretch level below. Terraform is a senior-role nice-to-have, *not* a requirement — zero shame in skipping it this round.

STRETCH LEVEL · WEEK 7

STRETCH

80 pts

*Make the whole system reproducible from a blank Azure account with one command.*

### ① LEARN FIRST · ~1–2 SESSIONS

**Grasp:** declarative vs imperative infrastructure; the **plan** → **apply** cycle; what **state** is and why it's sacred; providers, resources, and modules.

**Study:** HashiCorp Learn — "What is Terraform" & "Intro to IaC"

Terraform — Azure provider getting-started

**Ready to build when you can explain:** what Terraform state is, and why clicking around the Azure portal after `apply` breaks everything.

### ② THEN BUILD

- ☐ Learn Terraform core: providers, resources, state, modules. +30
- ☐ Write a script that provisions your Azure ML + AKS environment from scratch. +30
- ☐ Add a **GitHub Actions** CI/CD pipeline: push → build → push to ACR → deploy. +20

**Boss +40:** `terraform destroy` then `terraform apply` → everything rebuilds identically. Prove it.

**Tools:** Terraform   GitHub Actions   HashiCorp Learn

**Understanding test:** Why is Terraform state the most dangerous file in the repo? What does IaC give you that the Azure portal never will?

*Zero new learning. Turn the working prototype into a CV-ready solo project anyone can understand.*

-  One clean **README**: model → K8s → Azure → monitoring (→ RAG → Terraform if done). +35
-  One **architecture diagram** of the whole system. +25
-  Update **CV + LinkedIn** with skills, the project, and the AZ-900 badge. +40

**Optional (no points):** Record a 5-min walkthrough video. Not for demoing — purely interview rehearsal for "walk me through a system you built." Skip freely if short on time.

**Hint:** The README *is* the deliverable. Write it so a stranger could clone the repo and grasp the full pipeline in two minutes. That clarity is what makes it read as a real project on your CV, not a tutorial.

**Understanding test:** Can you whiteboard the entire system from memory and defend every tool choice? If yes — you're interview-ready.

# Scoreboard & Win Conditions

| Tier  | What you completed               | Points | Verdict                                                  |
|-------|----------------------------------|--------|----------------------------------------------------------|
| FLOOR | Kubernetes + AZ-900 + Grafana    | ~440   | CV transformed. A genuine win — not a consolation prize. |
| CORE  | All core levels (+ RAG/Langfuse) | 660    | Strong, well-rounded "ships production AI" profile.      |
| FULL  | + Terraform/CI-CD + all bosses   | 970    | Senior-signal portfolio. Now start CKA.                  |

Points map: Core base 660 · Stretch (Terraform) 80 · Bosses 230 → **970 max**. Hitting the Floor is success; everything above it is bonus, not debt.

## Recovery Protocol — for the inevitable bad stretch

| Situation           | What you do                                                                                 |
|---------------------|---------------------------------------------------------------------------------------------|
| Missed 1–2 days     | Ignore it. Resume where you left off. No catch-up cramming.                                 |
| Missed a whole week | Slide into the next buffer week (5 or 7). You lose nothing.                                 |
| Derailed 2+ weeks   | Drop to Floor-only mode: K8s + AZ-900 + Grafana. Still a win.                               |
| Dread building up   | Do a 25-min "tiny session" — redeploy something that already works. Momentum over progress. |

The only real failure is silently quitting. Sliding, shrinking, and resting are all *inside* the plan.

## Certifications — When, Not Whether

- **AZ-900** → Boss of Level 2 (Week 3). Fast, recognised, study is baked into the week. €165. Get it.
- **CKA** → **Not during these 60 days**. Only after you've lived in Kubernetes a while. Start prep *after* the challenge — rushing it wastes €395.

## Skills On Your CV After This

Python

Docker

Kubernetes

Helm

Azure (AZ-900)

AKS

Azure ML

ACR

Prometheus

Grafana

RAG + Langfuse / LLMops

FastAPI

CI/CD

Terraform (stretch)

vLLM (boss)

Realistic outcome: **deployment-fluent and screening-ready** — you'll have credibly touched every layer and can speak to all of it. Senior-grade depth in any one comes from the job this lands you. Get past the filter, then deepen on the clock.

If at the end you can explain **why each layer exists** — not just how to run it — you have changed your skill set. The CV update is just the receipt.

Challenger: \_\_\_\_\_ · Start: \_\_\_\_\_ · Floor target: **Week 4** · Ship: \_\_\_\_\_